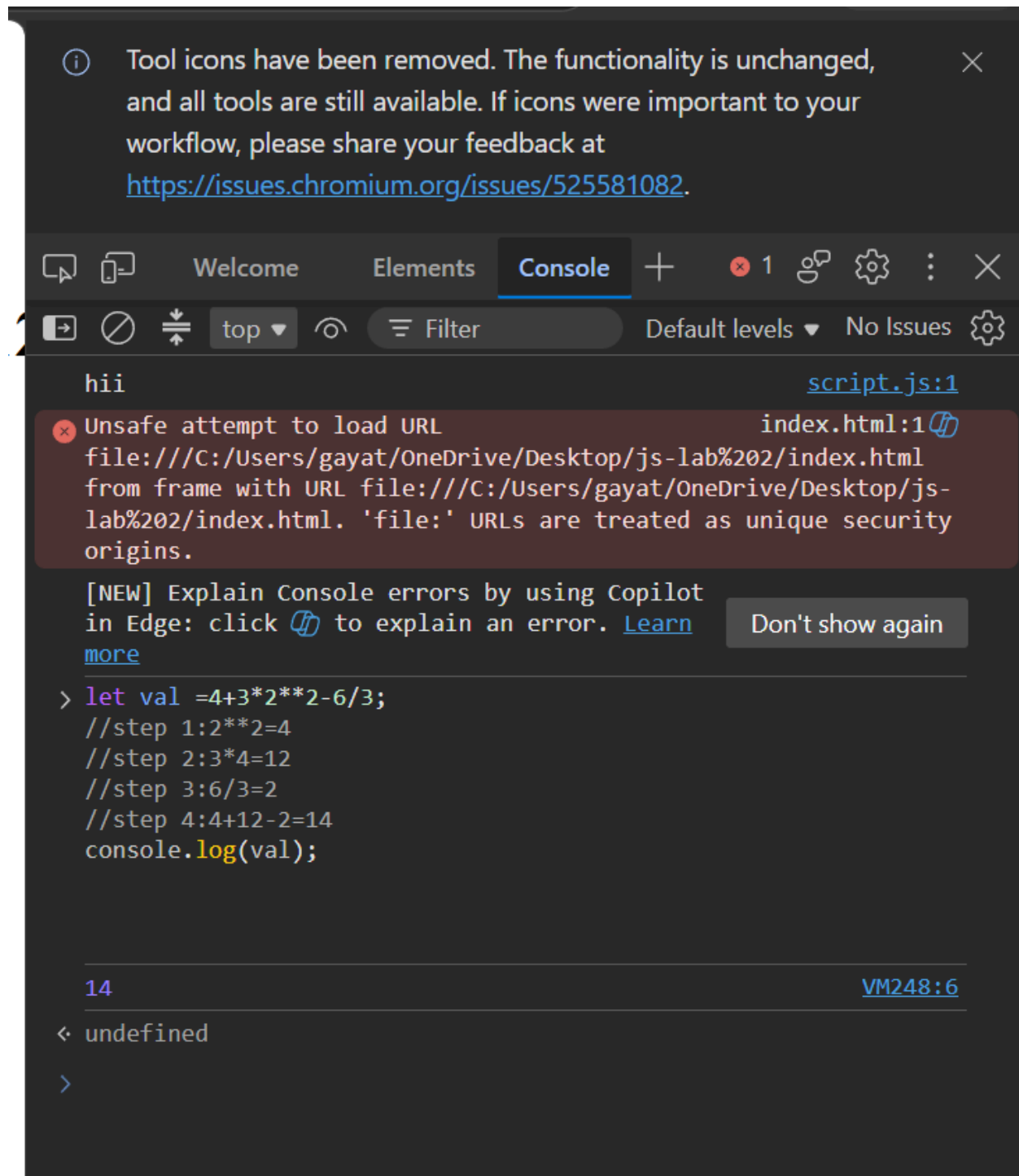
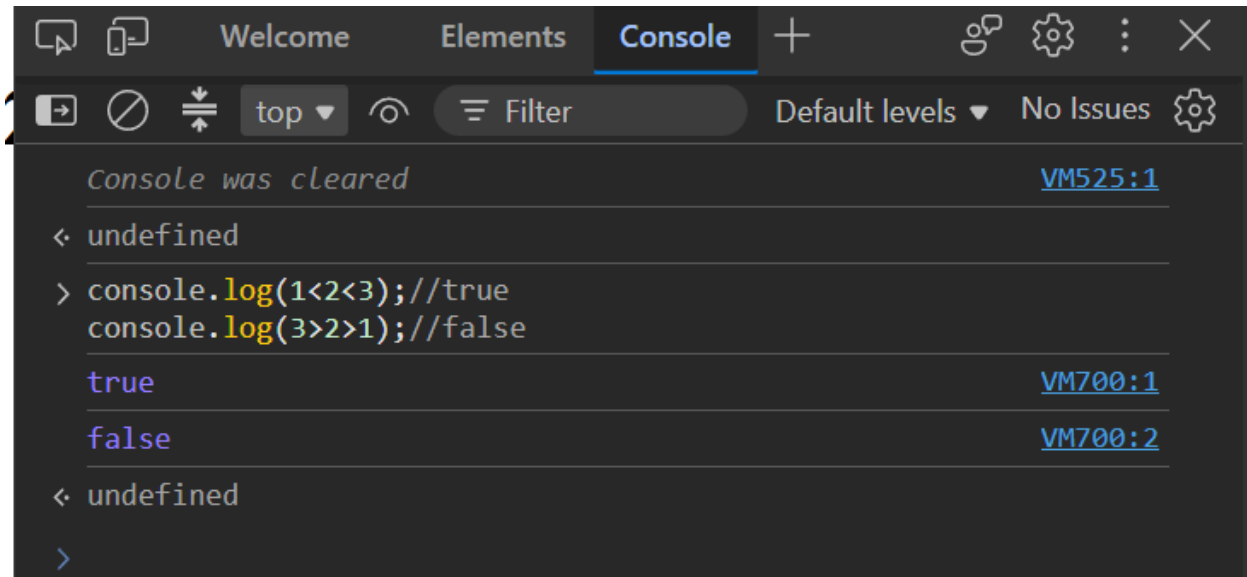


Part -1 :Operator Precedence

Task 1.1:



Task 1.2:



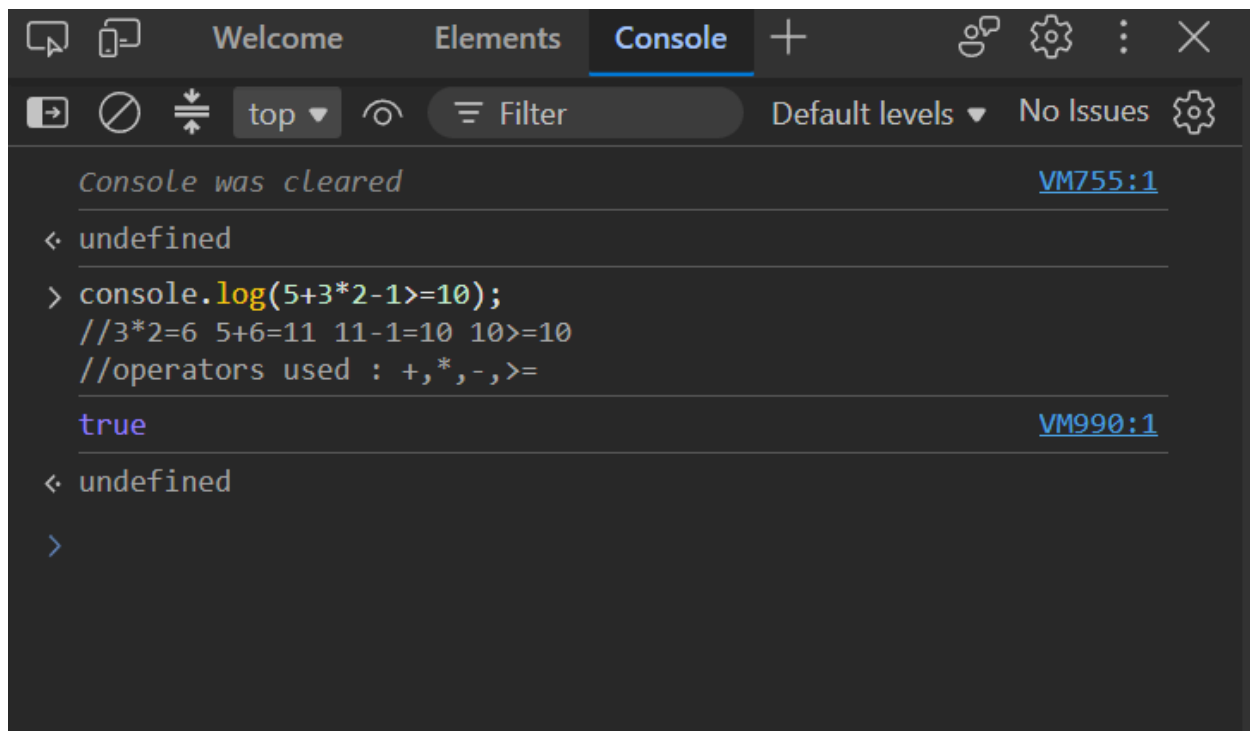
```

Welcome Elements Console
top Filter Default levels No Issues
Console was cleared VM525:1
< undefined
> console.log(1<2<3);//true
  console.log(3>2>1);//false
true VM700:1
false VM700:2
< undefined
>

```

Comparison operators are left-associative and evaluate pairwise, left to right — they don't chain mathematically. $1 < 2 < 3 \rightarrow (1 < 2)$ is true $\rightarrow \rightarrow$ true coerces to 1 $\rightarrow 1 < 3$ $1 < 3 \rightarrow$ true (works, coincidentally). $3 > 2 > 1 \rightarrow (3 > 2)$ is true $\rightarrow \rightarrow$ true coerces to 1 $\rightarrow 1 > 1 \rightarrow$ false .

Task 1.3: (4 operators, evaluates to true)



```

Welcome Elements Console
top Filter Default levels No Issues
Console was cleared VM755:1
< undefined
> console.log(5+3*2-1>=10);
  //3*2=6 5+6=11 11-1=10 10>=10
  //operators used : +,*,-,>=
true VM990:1
< undefined
>

```

Part 2 : Nested Ternary

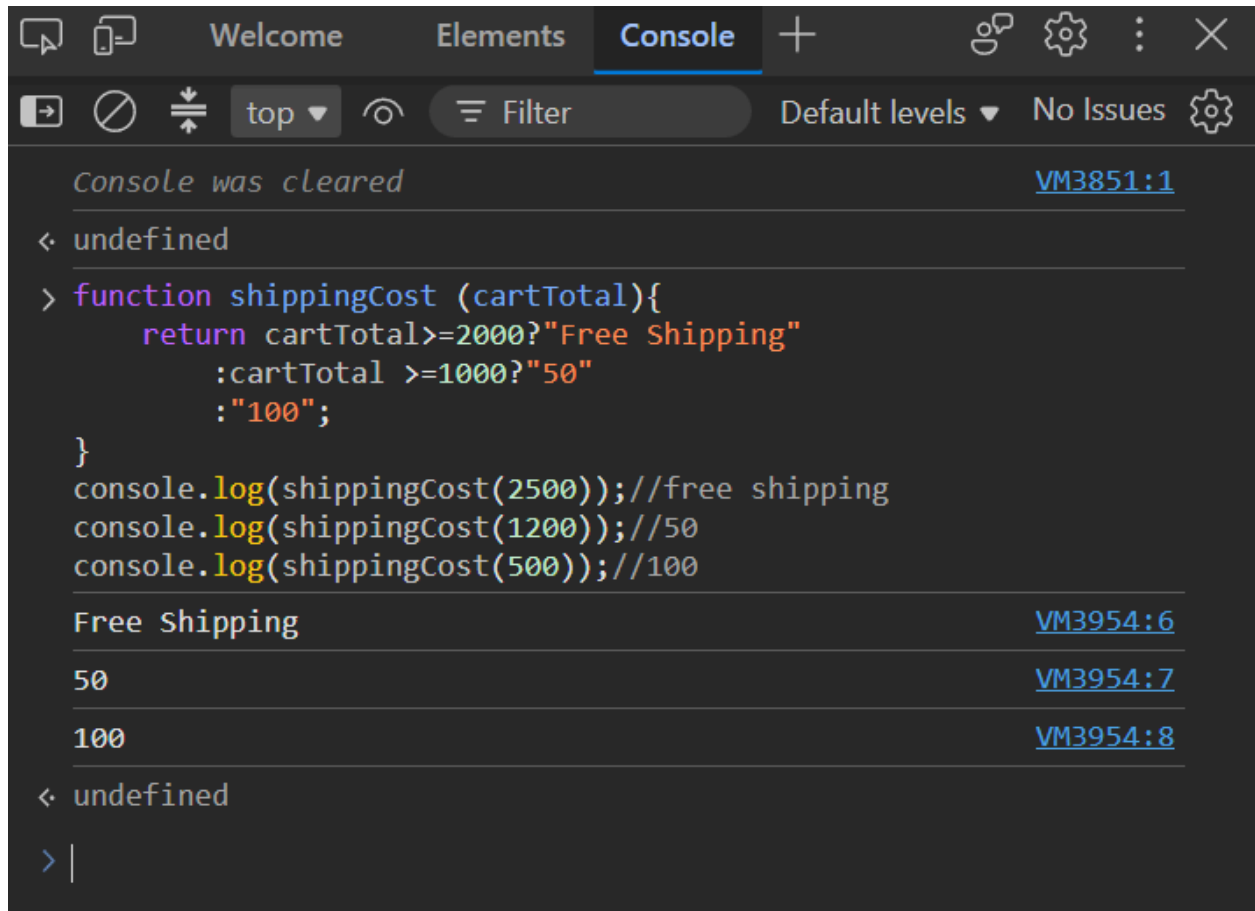
Task 2.1: Movie Ticket Pricing

```
< undefined
> function ticketPrice(age){
    return age <5? "Free"
      :age <12?"100"
      :age <60?"250"
      : "150";
}
console.log(ticketPrice(3));// Free
console.log(ticketPrice(10));//100
console.log(ticketPrice(25));//250
console.log(ticketPrice(45));//250
console.log(ticketPrice(65));//150
```

Free	VM1444:7
100	VM1444:8
250	VM1444:9
250	VM1444:10
150	VM1444:11

```
< undefined
> |
```

Task 2.2: Snipping Cost

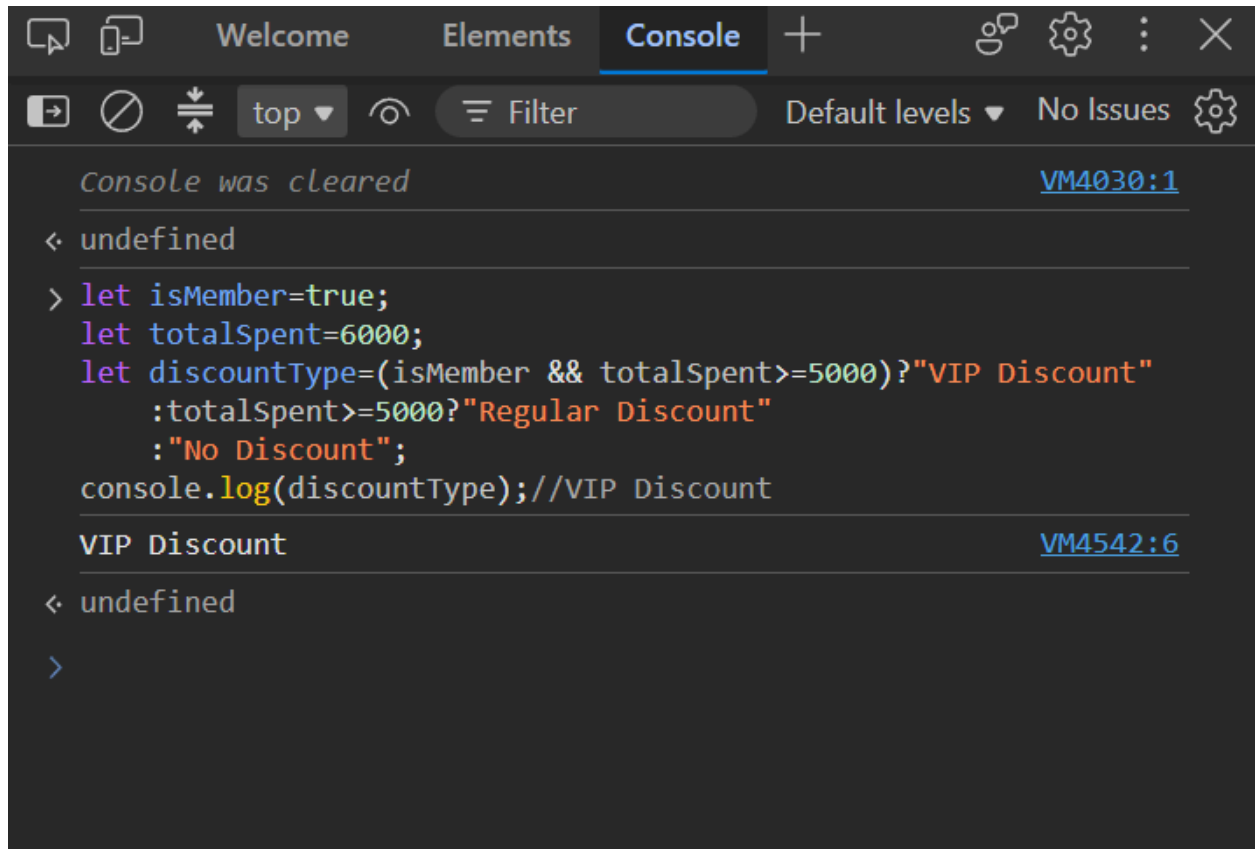


The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following content:

```
Console was cleared VM3851:1
< undefined
> function shippingCost (cartTotal){
    return cartTotal>=2000?"Free Shipping"
       :cartTotal >=1000?"50"
       : "100";
}
console.log(shippingCost(2500)); //free shipping
console.log(shippingCost(1200)); //50
console.log(shippingCost(500)); //100
Free Shipping VM3954:6
50 VM3954:7
100 VM3954:8
< undefined
> |
```

The console interface includes a toolbar at the top with icons for opening the console, clearing it, and other actions. The 'Filter' button is also visible. The log messages are color-coded: function definitions are in purple, return values in orange, and log output in green. Each log entry is followed by a link to the source code (e.g., VM3851:1, VM3954:6).

Task 2.3:



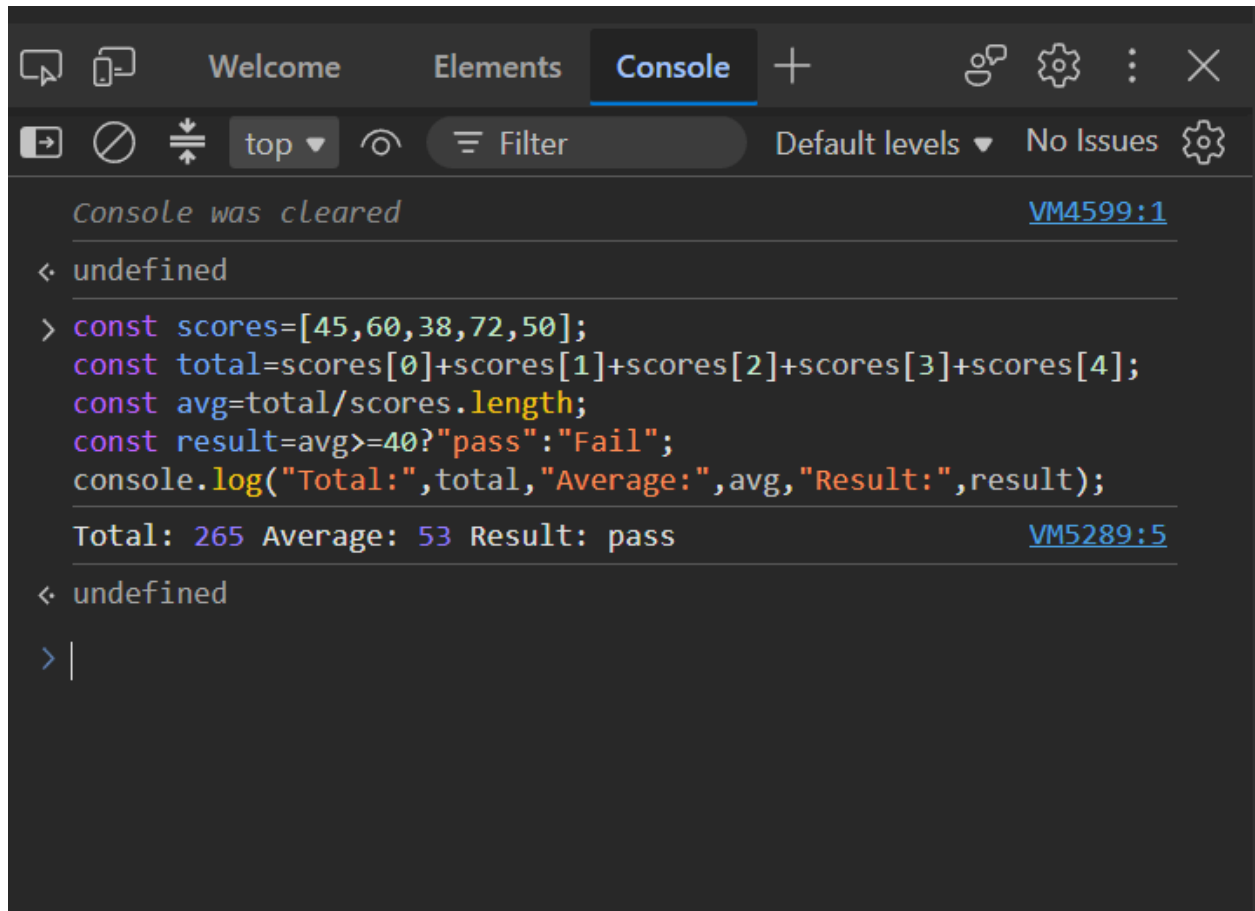
The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following content:

```
Console was cleared VM4030:1
< undefined
> let isMember=true;
let totalSpent=6000;
let discountType=(isMember && totalSpent>=5000)?"VIP Discount"
:totalSpent>=5000?"Regular Discount"
:"No Discount";
console.log(discountType);//VIP Discount
VIP Discount VM4542:6
< undefined
>
```

The code defines a variable `isMember` as `true` and `totalSpent` as `6000`. It then uses a ternary operator to assign `discountType` based on whether `isMember` is true and `totalSpent` is greater than or equal to 5000. The result is `"VIP Discount"`, which is logged to the console.

Part 3:Manual Aggregation

Task 3.1

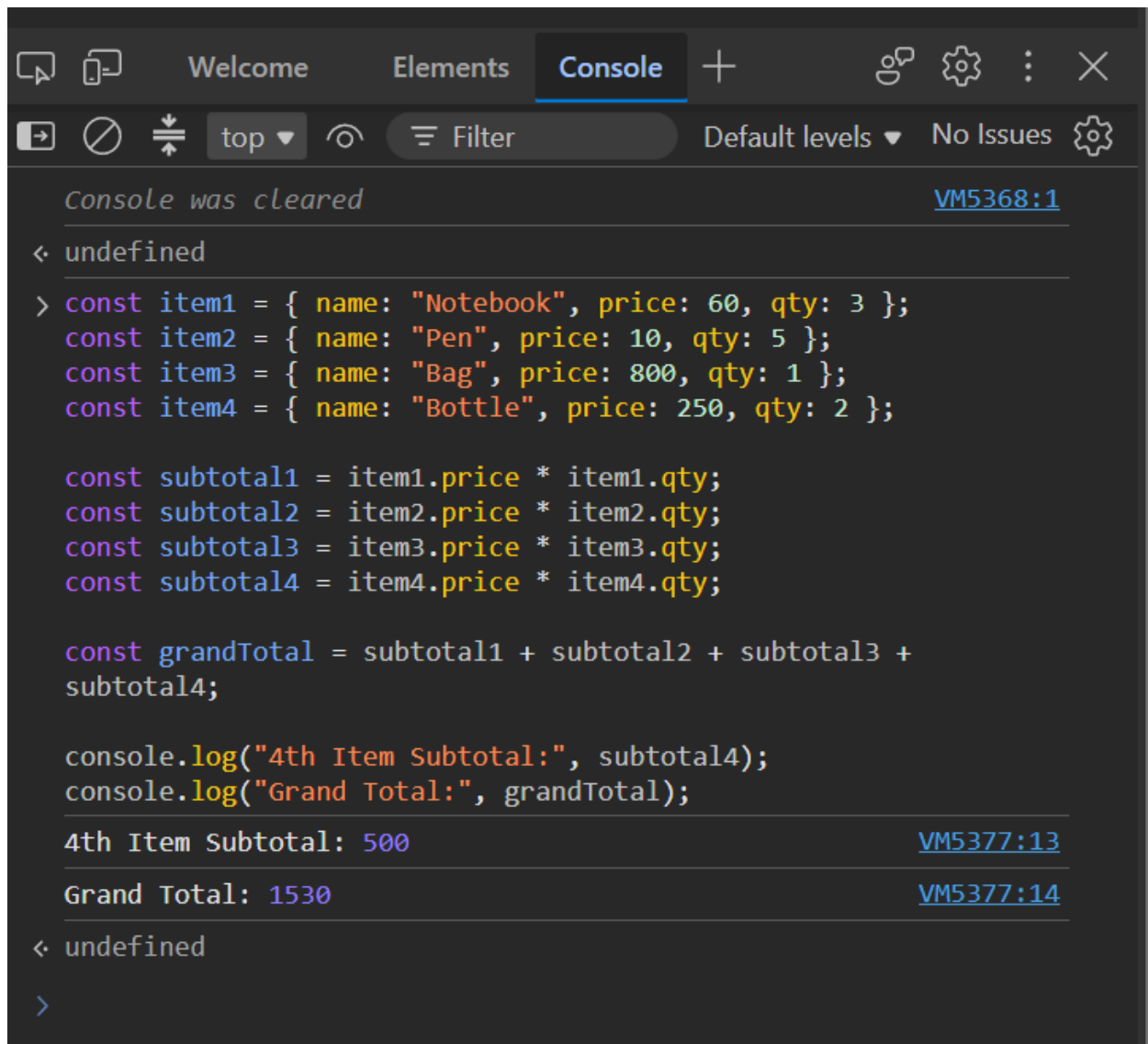


The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following sequence of events:

- A message: *Console was cleared* (source: [VM4599:1](#))
- An *undefined* value (source: [VM4599:1](#))
- Execution of the following JavaScript code:

```
> const scores=[45,60,38,72,50];  
const total=scores[0]+scores[1]+scores[2]+scores[3]+scores[4];  
const avg=total/scores.length;  
const result=avg>=40?"pass":"Fail";  
console.log("Total:",total,"Average:",avg,"Result:",result);
```
- The output of the `console.log` statement: *Total: 265 Average: 53 Result: pass* (source: [VM5289:5](#))
- Another *undefined* value (source: [VM5289:5](#))
- A prompt character `>` followed by a vertical bar `|`, indicating the next line of code is ready to be entered.

Task 3.2:



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its output:

```
Console was cleared VM5368:1
< undefined
> const item1 = { name: "Notebook", price: 60, qty: 3 };
const item2 = { name: "Pen", price: 10, qty: 5 };
const item3 = { name: "Bag", price: 800, qty: 1 };
const item4 = { name: "Bottle", price: 250, qty: 2 };

const subtotal1 = item1.price * item1.qty;
const subtotal2 = item2.price * item2.qty;
const subtotal3 = item3.price * item3.qty;
const subtotal4 = item4.price * item4.qty;

const grandTotal = subtotal1 + subtotal2 + subtotal3 +
subtotal4;

console.log("4th Item Subtotal:", subtotal4);
console.log("Grand Total:", grandTotal);
4th Item Subtotal: 500 VM5377:13
Grand Total: 1530 VM5377:14
< undefined
>
```

Part 4 — Deep Dive: Tricky Type Coercion

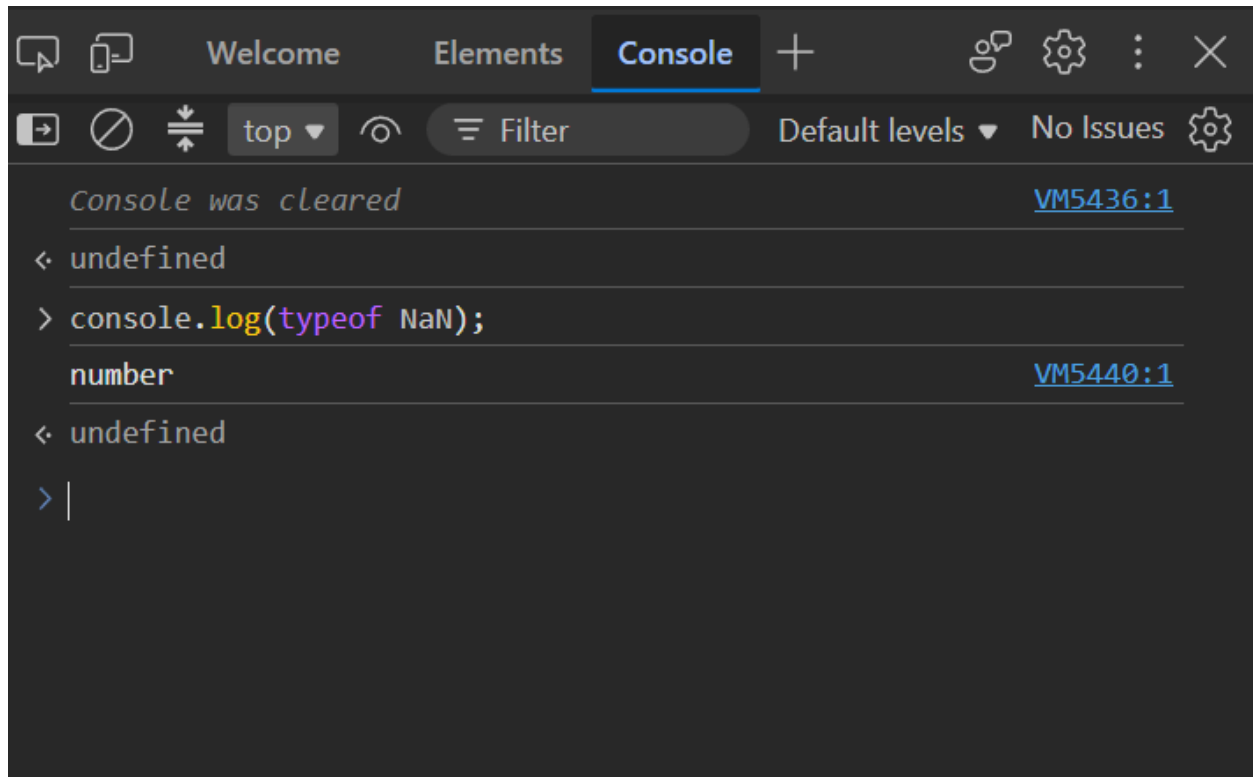
<u>N</u>	<u>Expression</u>	<u>Output</u>	<u>Reason</u>
<u>Q.</u>			
1	"5" + 3	"53"	+ joins the number with the string as text.

2	"5" - 3	2	- converts the string "5" into a number.
3	"abc" * 2	NaN	"abc" cannot be converted into a number.
4	NaN === NaN	false	NaN is not equal to itself.
5	[] == false	true	Loose equality converts both values before comparison.
6	"10" == 10	true	== converts the string "10" to the number 10.
7	null + 1	1	In numeric addition, null is converted to 0.

Most surprising result

NaN === NaN gives false. NaN means an invalid numeric result, and it has a special behavior in JavaScript. Therefore, even comparing NaN with another NaN using === gives false.

Task 4.2:



Reason: NaN is a special value belonging to JavaScript's number data type. It represents an invalid or unrepresentable numeric result.

Part 5 — Smart Shopping Cart & Billing System

```

Welcome Elements Console Sources Network Performance Memory Application Security Lighthouse
top Filter
Console was cleared
< undefined
> const item1 = {
  name: "Notebook",
  price: 600,
  qty: 3
};

const item2 = {
  name: "Headphones",
  price: 1200,
  qty: 1
};

const item3 = {
  name: "Backpack",
  price: 900,
  qty: 1
};

console.log(typeof item1.price);
console.log(typeof item2.price);
console.log(typeof item3.price);

const subtotal1 = item1.price * item1.qty;
const subtotal2 = item2.price * item2.qty;
const subtotal3 = item3.price * item3.qty;
```

```

Welcome Elements Console Sources Network Performance Memory Application Security Lighthouse +
top Filter Default levels No Issues
const subtotal1 = item1.price * item1.qty;

const grandTotal = subtotal1 + subtotal2 + subtotal3;

const discountRate = grandTotal >= 5000 ? 0.20
  : grandTotal >= 2000 ? 0.10
  : grandTotal >= 1000 ? 0.05
  : 0;

const discountAmount = grandTotal * discountRate;
const afterDiscount = grandTotal - discountAmount;

const gst = afterDiscount * 0.18;
const finalAmount = afterDiscount + gst;

const freeShipping = afterDiscount >= 1500 || 3 >= 3;
console.log("----- SHOPPING RECEIPT -----");
console.log(item1.name + ": ₹" + subtotal1);
console.log(item2.name + ": ₹" + subtotal2);
console.log(item3.name + ": ₹" + subtotal3);
console.log("Grand Total: ₹" + grandTotal);
console.log("Discount: " + (discountRate * 100) + "%");
console.log("Discount Amount: ₹" + discountAmount);
console.log("After Discount: ₹" + afterDiscount);
console.log("GST (18%): ₹" + gst);
console.log("Final Payable: ₹" + finalAmount);
console.log("Shipping: " + (freeShipping ? "FREE" : "₹100 shipping charge"));

number VM5562:19
number VM5562:20
```

```
console.log( Shipping: " + (freeshipping ? FREE : ₹100 Shipping Charge ));
number VM5562:19
number VM5562:20
number VM5562:21
----- SHOPPING RECEIPT ----- VM5562:41
Notebook: ₹1800 VM5562:42
Headphones: ₹1200 VM5562:43
Backpack: ₹900 VM5562:44
Grand Total: ₹3900 VM5562:45
Discount: 10% VM5562:46
Discount Amount: ₹390 VM5562:47
After Discount: ₹3510 VM5562:48
GST (18%): ₹631.8 VM5562:49
Final Payable: ₹4141.8 VM5562:50
Shipping: FREE VM5562:51
undefined
>
```

Bonus loyalty points:

```
Welcome Elements Console Sources Network Performance Memory Application Security Lighthouse +
top Filter
Console was cleared VM5625:1
< undefined
> const loyaltyPoints = finalAmount / 100;
  console.log("Loyalty Points:", loyaltyPoints);
  Loyalty Points: 41.418 VM5629:2
< undefined
> |
```

Part 6 — Debugging Challenge

Snippet 1:

```
Welcome Elements Console Sources Network Performance Memory Application Security Lighthouse +
top Filter
Console was cleared VM5688:1
< undefined
> let price = 500;
  let discount = price * 0.1;
  discount = discount + 5;

  console.log("Final: " + discount);
  Final: 55 VM5692:5
< undefined
> |
```

Bugs

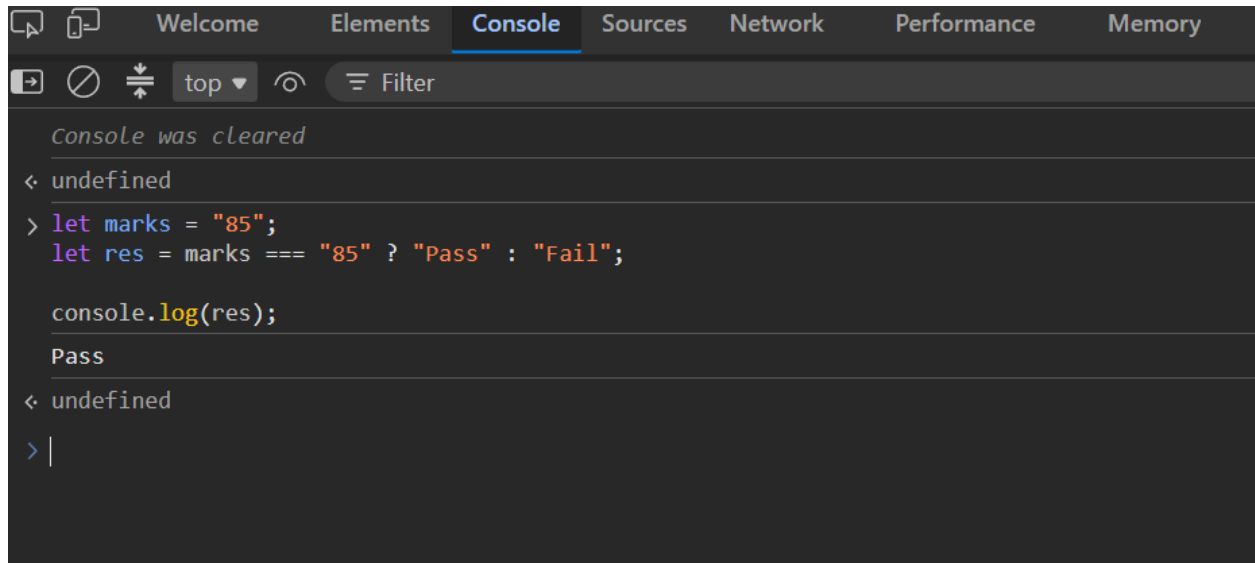
Bug 1: discount was declared using const, but its value was changed later.

Correction: Changed const to let.

Bug 2: final was not declared.

Correction: Changed final to discount.

Snippet 2 :



```

Welcome Elements Console Sources Network Performance Memory
top Filter
Console was cleared
< undefined
> let marks = "85";
let res = marks === "85" ? "Pass" : "Fail";

console.log(res);
Pass
< undefined
> |
```

Bugs

Bug 1: marks contains the string "85", but the code compares it with the number 85 using ===.

Correction: Compared "85" with "85".

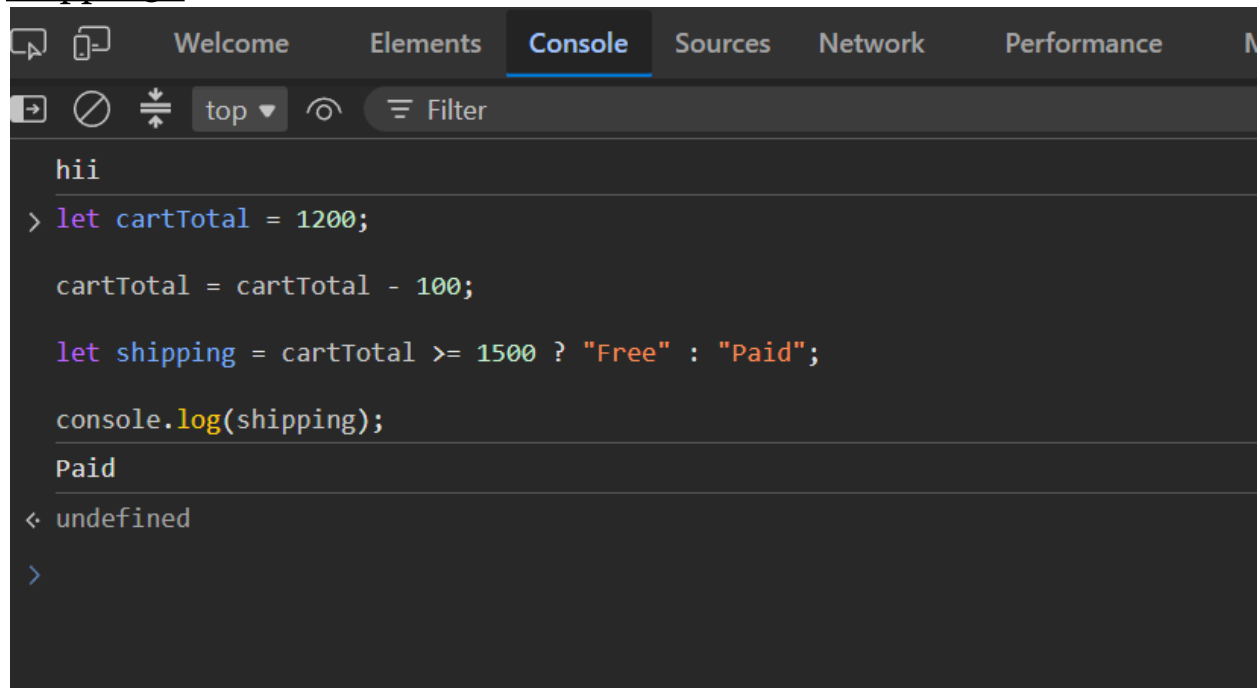
Bug 2: Result and result are different names because JavaScript is case-sensitive.

Correction: Changed Result to result.

Why did the === bug not produce an error?

Because comparing different data types with === is valid JavaScript. It simply returns false; therefore, the program continues and gives the wrong logical result instead of throwing an error.

Snippet 3 :

A screenshot of a web browser's developer console. The console has tabs for Welcome, Elements, Console (selected), Sources, Network, and Performance. Below the tabs is a toolbar with icons for opening the console, clearing it, and a filter button. The console shows the following code and output:

```
hii
> let cartTotal = 1200;

cartTotal = cartTotal - 100;

let shipping = cartTotal >= 1500 ? "Free" : "Paid";

console.log(shipping);
Paid
< undefined
>
```

Bugs

Bug 1: cartTotal was declared using const, but its value was changed later.

Correction: Changed const to let.

Bug 2: The ternary expression was missing the : between "Free" and "Paid".

Correction is

```
cartTotal >= 1500 ? "Free" : "Paid";
```

Short Paragraph — Most Difficult Part :

The part that pushed me the most was the tricky type coercion section.

Some JavaScript expressions gave unexpected results, especially NaN === NaN and [] == false. It helped me understand that JavaScript sometimes converts values automatically depending on the operator being used.

